

Práctica: SEGG-U1- P1OWASP Top 10: Investigación y análisis de vulnerabilidades

Universidad Tecnológica de Querétaro

Diego Jesus Reyes Rebolledo

2024171017 20 de mayo de 2026

Introducción

El OWASP Top 10 es una guía de referencia utilizada por desarrolladores, empresas y equipos de seguridad para identificar los riesgos más importantes en aplicaciones web. Su objetivo principal es crear conciencia sobre las vulnerabilidades más críticas y ayudar a que los sistemas se desarrollen con mejores prácticas desde el inicio.

En esta práctica se analizará la versión OWASP Top 10:2025, ya que es la versión más actual publicada por OWASP. Aunque la guía de la actividad menciona OWASP Top 10 2023, se decidió trabajar con la versión vigente para usar información más reciente y alineada con los riesgos actuales de seguridad.

El reporte incluye una tabla comparativa de las diez categorías principales, ejemplos de ataque, posibles impactos, métodos de mitigación, casos reales documentados y ejemplos de código vulnerable comparado con código seguro. El propósito no es aprender a atacar sistemas, sino comprender cómo ocurren estas fallas para prevenirlas durante el desarrollo de software.

Código	Vulnerabilidad
A01:2025	Broken Access Control
A02:2025	Security Misconfiguration
A03:2025	Software Supply Chain Failures
A04:2025	Cryptographic Failures
A05:2025	Injection
A06:2025	Insecure Design
A07:2025	Authentication Failures
A08:2025	Software or Data Integrity Failures
A09:2025	Security Logging and Alerting Failures
A10:2025	Mishandling of Exceptional Conditions

Tabla comparativa OWASP Top 10

La siguiente tabla presenta las diez categorías del OWASP Top 10:2025. Para cada vulnerabilidad se incluye una descripción breve, un ejemplo de ataque, el impacto que podría causar en una aplicación web y una medida general de mitigación. La información se redactó con base en la documentación oficial de OWASP, evitando reproducir exploits activos y enfocándose en el aprendizaje preventivo.

Código	Vulnerabilidad	Descripción corta	Ejemplo de ataque	Impacto potencial	Método de mitigación
A01:2025	Broken Access Control	Ocurre cuando el sistema no controla correctamente qué puede hacer cada usuario. OWASP menciona ejemplos como modificar URLs, acceder a cuentas ajenas o usar APIs sin permisos adecuados.	Un usuario cambia la URL de <code>/perfil/10</code> a <code>/perfil/11</code> y logra ver información de otra persona.	Robo de datos, modificación de información, eliminación de registros o acceso a funciones administrativas.	Validar permisos desde el servidor, negar accesos por defecto, aplicar roles correctamente y registrar intentos fallidos de acceso.
A02:2025	Security Misconfiguration	Sucede cuando una aplicación, servidor, base de datos o servicio en la nube está mal configurado desde el punto de vista de seguridad. OWASP menciona errores como cuentas por defecto, servicios innecesarios y mensajes de error demasiado detallados.	Un servidor deja activo un panel de administración con usuario y contraseña por defecto.	Acceso no autorizado, exposición de información interna o toma de control parcial del sistema.	Eliminar funciones innecesarias, cambiar credenciales por defecto, revisar permisos, ocultar errores técnicos y automatizar configuraciones seguras.
A03:2025	Software Supply Chain Failures	Se refiere a fallas en la	Un desarrollador	Robo de credenciales,	Controlar versiones, usar fuentes confiables, revisar

		cadena de suministro del software, como dependencias, librerías, herramientas o procesos externos comprometidos. OWASP señala que puede involucrar componentes vulnerables, sin mantenimiento o de fuentes no confiables.	instala una librería de npm comprometida que roba variables de entorno o tokens del proyecto.	distribución de código malicioso, compromiso de usuarios y afectación de sistemas completos.	dependencias, aplicar actualizaciones, separar responsabilidades y monitorear componentes externos.
A04:2025	Cryptographic Failures	Aparece cuando los datos sensibles no se protegen correctamente con cifrado, hashing seguro o buena gestión de llaves. OWASP menciona problemas como algoritmos débiles, llaves expuestas, falta de TLS y hashes inseguros.	Una aplicación guarda contraseñas con MD5 o transmite datos personales por HTTP sin cifrado.	Exposición de contraseñas, datos personales, información bancaria o secretos de negocio.	Usar TLS actualizado, cifrar datos sensibles, no guardar datos innecesarios, usar hashing fuerte con salt y evitar algoritmos obsoletos como MD5 o SHA1.
A05:2025	Injection	Ocurre cuando datos ingresados por el usuario se envían directamente a un intérprete como SQL, comandos del sistema o navegador, permitiendo que se ejecuten	En un login, el atacante escribe ' OR '1'='1 para alterar una consulta SQL y entrar sin credenciales válidas.	Robo, modificación o eliminación de datos; acceso no autorizado; ejecución de comandos en el servidor.	Usar consultas parametrizadas, validar entradas en servidor, evitar concatenar datos en consultas y aplicar pruebas SAST/DAST en el pipeline.

		como instrucciones. (owasp.org)			
A06:2025	Insecure Design	No es solo un error de código, sino una falla de diseño. OWASP explica que ocurre cuando faltan controles de seguridad desde la arquitectura o la lógica del negocio.	Una tienda en línea permite aplicar cupones ilimitados porque nunca se diseñó una regla para limitar su uso.	Fraudes, abuso de funciones, pérdida económica o fallas lógicas difíciles de corregir después.	Hacer modelado de amenazas, definir requisitos de seguridad, usar patrones de diseño seguro y revisar la lógica del negocio antes de programar. (owasp.org)
A07:2025	Authentication Failures	Se presenta cuando el sistema permite que un atacante sea reconocido como usuario legítimo. OWASP incluye problemas como contraseñas débiles, falta de MFA, sesiones mal manejadas y ataques automatizados.	Un atacante usa credential stuffing con correos y contraseñas filtradas de otras plataformas.	Secuestro de cuentas, acceso a información personal, fraude o escalamiento de privilegios.	Implementar MFA, bloquear ataques automatizados, evitar credenciales por defecto, validar contraseñas contra listas filtradas y manejar sesiones de forma segura.
A08:2025	Software or Data Integrity Failures	Ocurre cuando una aplicación confía en software, actualizaciones, datos serializados o componentes sin verificar su integridad. OWASP menciona firmas digitales, CI/CD inseguro y deserialización insegura.	Una aplicación instala una actualización automática sin verificar firma digital y termina ejecutando código malicioso.	Ejecución de código no autorizado, alteración de datos, secuestro de sesiones o compromiso del sistema.	Verificar firmas digitales, proteger el pipeline CI/CD, controlar cambios, usar repositorios confiables y evitar datos serializados sin validación de integridad.

A09:2025	Security Logging and Alerting Failures	Sucede cuando el sistema no registra, monitorea o alerta correctamente eventos importantes de seguridad. OWASP señala que sin logs y alertas es difícil detectar ataques o responder a incidentes.	Un atacante intenta iniciar sesión cientos de veces, pero el sistema no registra ni alerta esos intentos fallidos.	Detección tardía de ataques, pérdida de evidencia, mayor impacto en incidentes y dificultad para investigar.	Registrar eventos importantes, proteger logs contra alteración, generar alertas útiles, monitorear actividad sospechosa y definir planes de respuesta.
A10:2025	Mishandling of Exceptional Conditions	Es una categoría nueva en 2025. OWASP la relaciona con mal manejo de errores, fallas lógicas, condiciones inesperadas, errores no controlados y situaciones donde el sistema “falla abierto”.	Una API lanza un error y muestra información interna como rutas, nombres de tablas, tokens o detalles del servidor.	Filtración de información sensible, caídas del sistema, comportamiento inesperado o fallas de autorización.	Manejar excepciones correctamente, mostrar mensajes seguros al usuario, registrar errores internos, usar manejadores globales y monitorear patrones de errores.

Casos reales por vulnerabilidad

A continuación se presentan casos reales relacionados con las categorías del OWASP Top 10:2025. Los casos fueron seleccionados a partir de fuentes públicas y verificables, como OWASP, CISA, NVD, CVE.org, comunicados oficiales de empresas y reportes técnicos reconocidos. La finalidad de esta sección no es reproducir ataques, sino entender cómo fallas de seguridad reales han afectado a organizaciones y qué tipo de impacto pueden generar.

A01:2025 — Broken Access Control

OWASP define esta categoría como fallas donde los usuarios pueden actuar fuera de los permisos que deberían tener, lo cual puede causar acceso no autorizado, modificación o eliminación de información.

Caso real	Empresa / producto afectado	Año	Cómo fue explotada	Relación con OWASP
CVE-2023-22515	Atlassian Confluence Data Center y Server	2023	Atacantes externos explotaron una falla de control de acceso en instancias públicas de Confluence para crear cuentas de administrador no autorizadas y acceder al sistema. CISA la describe como una vulnerabilidad crítica de Broken Access Control. (cisa.gov)	Es un caso directo de acceso no autorizado por falla en controles de permisos.
CVE-2023-20198	Cisco IOS XE Web UI	2023	La vulnerabilidad permitía a un atacante obtener privilegios administrativos en dispositivos Cisco IOS XE con la interfaz web HTTP/HTTPS habilitada. CISA indica que la explotación podía dar acceso administrativo completo al sistema. (cisa.gov)	Se relaciona con control de acceso roto porque un actor sin permisos podía elevar privilegios y controlar el dispositivo.

A02:2025 — Security Misconfiguration

OWASP define esta categoría como errores donde una aplicación, servidor, servicio en la nube o base de datos queda mal configurada desde seguridad.

Caso real	Empresa / producto afectado	Año	Cómo fue explotada	Relación con OWASP
-----------	-----------------------------	-----	--------------------	--------------------

Capital One Data Breach	Capital One	2019	El Departamento de Justicia de EE. UU. señaló que la intrusión ocurrió por medio de un web application firewall mal configurado, lo que permitió acceso a datos almacenados por Capital One. (Departamento de Justicia)	Es un ejemplo claro de mala configuración en infraestructura de seguridad.
Microsoft Power Apps data exposure	Microsoft Power Apps / organizaciones usuarias	2021	Investigadores de UpGuard reportaron que configuraciones por defecto o mal entendidas en portales de Power Apps dejaron expuestos alrededor de 38 millones de registros, incluyendo datos sensibles de distintas organizaciones. (upguard.com)	Se relaciona con configuración insegura, especialmente permisos y exposición pública de datos.

A03:2025 — Software Supply Chain Failures

Caso real	Empresa / producto afectado	Año	Cómo fue explotada	Relación con OWASP
SolarWinds Orion compromise	SolarWinds Orion	2020	MITRE describe que atacantes inyectaron código malicioso en el proceso de construcción de SolarWinds Orion y ese código fue distribuido mediante actualizaciones legítimas del software. (attack.mitre.org)	Es uno de los ejemplos más fuertes de ataque a la cadena de suministro de software.
3CX Desktop App compromise	3CX Desktop App	2023	Mandiant reportó que una versión comprometida del software legítimo de 3CX distribuyó malware. Además, el origen estuvo relacionado con otro software comprometido previamente, X_TRADER, lo que hizo el caso especialmente grave. (Google Cloud)	Es un caso de software confiable que terminó distribuyendo código malicioso a sus usuarios.

A04:2025 — Cryptographic Failures

Caso real	Empresa / producto afectado	Año	Cómo fue explotada	Relación con OWASP

Adobe Data Breach	Adobe	2013	Adobe informó que atacantes accedieron a IDs de clientes, contraseñas cifradas y datos relacionados con tarjetas. El análisis posterior de Have I Been Pwned señala que la criptografía de contraseñas fue débil y que las pistas de contraseña estaban en texto plano. (Adobe Blog)	Se relaciona con protección criptográfica deficiente de credenciales y datos sensibles.
LinkedIn password breach	LinkedIn	2012 / revelado ampliamente en 2016	LinkedIn confirmó que datos de 2012 incluían correos, IDs internos y contraseñas hasheadas. Have I Been Pwned documenta que las contraseñas estaban protegidas con SHA-1 sin salt, lo que facilitó que muchas fueran crackeadas. (LinkedIn)	Es un caso típico de hashing inseguro de contraseñas.

A05:2025 — Injection

OWASP define injection como una falla donde entradas no confiables del usuario llegan a un intérprete, como SQL, comandos del sistema o navegador, y se ejecutan como instrucciones.

Caso real	Empresa / producto afectado	Año	Cómo fue explotada	Relación con OWASP
CVE-2023-34362	Progress MOVEit Transfer	2023	NVD documenta que MOVEit Transfer tenía una vulnerabilidad de SQL Injection que podía permitir a un atacante no autenticado acceder a la base de datos de MOVEit Transfer. (NVD)	Es un caso directo de inyección SQL con impacto real en datos.
CVE-2021-44228 / Log4Shell	Apache Log4j	2021	CVE.org indica que un atacante que controlara mensajes o parámetros de log podía provocar ejecución de código remoto mediante servidores LDAP. CISA también describe Log4Shell como una vulnerabilidad RCE en Apache Log4j. (CVE)	Es un caso de inyección donde una entrada manipulada termina ejecutando código no autorizado.

A06:2025 — Insecure Design

OWASP define **Insecure Design** como fallas relacionadas con decisiones de diseño, arquitectura o lógica de negocio, no solamente errores de programación. La prevención se enfoca en modelado de amenazas, patrones seguros y requisitos de seguridad desde el diseño.

Caso real	Empresa / producto afectado	Año	Cómo fue explotada o aprovechada	Relación con OWASP
Facebook “View As”	Facebook / Meta	2018	Atacantes explotaron una vulnerabilidad en la función “View As”, lo que les permitió robar access tokens. Facebook informó que reinició tokens de casi 50 millones de cuentas afectadas y otros 40 millones de forma preventiva. (Sobre Facebook)	Se relaciona con diseño inseguro porque una función de privacidad terminó interactuando mal con tokens de sesión, generando un riesgo grave desde la lógica de la plataforma.
Venmo privacy/security practices	Venmo / PayPal	2018	La FTC señaló que Venmo no comunicaba claramente cómo funcionaban sus controles de privacidad y también cuestionó afirmaciones de “bank-grade security”. Además, la FTC indicó que Venmo no notificaba correctamente cambios sensibles como contraseña, correo o nuevo dispositivo. (Federal Trade Commission)	Es un caso de diseño inseguro porque los controles de privacidad y seguridad no estaban planteados de forma suficientemente clara ni segura para proteger al usuario desde el diseño del servicio.

A07:2025 — Authentication Failures

OWASP mantiene esta categoría para fallas relacionadas con autenticación, sesiones, credenciales débiles, credenciales reutilizadas, MFA ausente o mal manejo de sesiones.

Caso real	Empresa / producto afectado	Año	Cómo fue explotada o aprovechada	Relación con OWASP
23andMe credential stuffing	23andMe	2023	El ataque se basó en credenciales reutilizadas por usuarios. Autoridades de privacidad señalaron que 23andMe tardó en implementar medidas como MFA obligatorio y reset de sesiones, mientras el ataque basado en credenciales podía seguir activo. (priv.gc.ca)	Es un ejemplo directo de fallas de autenticación: credenciales reutilizadas, falta de MFA obligatorio desde el inicio y controles insuficientes contra credential stuffing.
Okta Support Case Management breach	Okta	2023	Okta informó que un actor no autorizado accedió a archivos del sistema de soporte usando una cuenta de servicio. Algunos archivos HAR contenían tokens de sesión que podían usarse para secuestro de sesión. (sec.okta.com)	Se relaciona con autenticación y manejo de sesiones porque el acceso dependió de credenciales comprometidas y los archivos expuestos contenían tokens utilizables para atacar sesiones.

A08:2025 — Software or Data Integrity Failures

OWASP explica que esta categoría aparece cuando una aplicación confía en código, actualizaciones, datos serializados o componentes sin verificar suficientemente su integridad. Incluye casos como actualizaciones inseguras y deserialización insegura.

Caso real	Empresa / producto afectado	Año	Cómo fue explotada o aprovechada	Relación con OWASP
CCleaner compromised build	CCleaner / Piriform / Avast	2017	Cisco Talos identificó que el instalador legítimo de CCleaner v5.33, distribuido desde servidores oficiales, contenía malware. (Cisco Talos Blog)	Es un caso fuerte de falla de integridad porque los usuarios confiaban en un instalador oficial que había sido alterado maliciosamente.
CVE-2017-9805 Apache Struts REST Plugin	Apache Struts	2017	NVD documenta que el plugin REST de Apache Struts usaba XStream para deserializar XML sin filtrado de tipos, lo que podía llevar a ejecución remota de código. (NVD)	Se relaciona con integridad de datos porque la aplicación confiaba en datos serializados modificables por el atacante.

A09:2025 — Security Logging and Alerting Failures

OWASP indica que los eventos importantes, como fallos de login, control de acceso e input validation, deben registrarse con suficiente contexto y generar alertas útiles. También aclara que los logs sin alertas tienen poco valor para identificar incidentes.

Caso real	Empresa / producto afectado	Año	Cómo fue explotada o aprovechada	Relación con OWASP
Target data breach	Target	2013	Un análisis del Senado de EE. UU. indicó que Target aparentemente no respondió a múltiples advertencias automáticas de su software anti-intrusión, tanto sobre instalación de malware como sobre rutas de exfiltración. (commerce.senate.gov)	Es un caso claro de falla de monitoreo y alertamiento: las alertas existieron, pero no provocaron una respuesta efectiva.
Equifax data breach	Equifax	2017	El reporte del Congreso de EE. UU. señala que el ataque duró 76 días. También se documentó que problemas como un certificado expirado afectaron la capacidad de inspección/detección del tráfico malicioso. (oversight.house.gov)	Se relaciona con logging y alerting porque la detección fue tardía y los controles de monitoreo no funcionaron adecuadamente durante el incidente.

A10:2025 — Mishandling of Exceptional Conditions

OWASP explica que esta categoría es nueva en 2025 y se enfoca en manejo incorrecto de errores, errores lógicos, fallas tipo “fail open” y otros escenarios provocados por condiciones anormales o inesperadas.

Caso real	Empresa / producto afectado	Año	Cómo fue explotada o aprovechada	Relación con OWASP
CVE-2023-29017 vm2 sandbox escape	vm2, librería de sandbox para Node.js	2023	NVD documenta que vm2 no manejaba correctamente objetos del host pasados a <code>Error.prepareStackTrace</code> durante errores async no manejados, permitiendo escapar del sandbox y lograr ejecución remota de código. (NVD)	Es un ejemplo muy directo de A10 porque el problema nace del mal manejo de una condición excepcional: errores async no controlados.
CVE-2025-13596 ATISoluciones CIGES	ATISoluciones CIGES	2025	NVD describe que, ante condiciones inesperadas que provocan excepciones no manejadas, la aplicación devuelve mensajes detallados y stack traces al cliente, exponiendo rutas internas, consultas SQL o detalles de configuración. (NVD)	Es un caso directo de manejo incorrecto de excepciones, porque el sistema revela información sensible cuando ocurre un error inesperado.

Estos casos muestran que las vulnerabilidades del OWASP Top 10 no son problemas teóricos. En la práctica, han afectado a empresas grandes, plataformas populares, librerías de desarrollo y servicios usados por millones de personas. También se puede observar que no todos los incidentes ocurren por una sola línea de código mal escrita; muchos nacen de malas decisiones de diseño, configuraciones débiles, falta de monitoreo, manejo incorrecto de sesiones o confianza excesiva en componentes externos.

Código vulnerable vs código seguro

En esta sección se muestran tres ejemplos de código vulnerable y su versión corregida. Los ejemplos están escritos en JavaScript de forma simplificada para explicar el problema de manera clara. No representan sistemas completos, sino fragmentos diseñados para comparar malas prácticas contra soluciones más seguras.

1. A01:2025 — Broken Access Control

Código vulnerable

```
// Ejemplo vulnerable: cualquier usuario puede consultar cualquier perfil
app.get('/usuarios/:id', async (req, res) => {
  const idUsuario = req.params.id;

  const usuario = await db.query(
    `SELECT * FROM usuarios WHERE id = ${idUsuario}`
  );

  res.json(usuario);
});
```

El problema es que el sistema permite consultar cualquier usuario solo cambiando el ID en la URL. Por ejemplo, si un usuario entra a /usuarios/5, podría cambiarlo a /usuarios/6 y ver información que no le pertenece. El error principal es que no se valida si el usuario autenticado realmente tiene permiso para acceder a ese recurso.

Código seguro

```
// Ejemplo seguro: el usuario solo puede consultar su propio perfil
app.get('/usuarios/:id', verificarSesion, async (req, res) => {

  const idSolicitado = Number(req.params.id);
  const idUsuarioSesion = req.usuario.id;

  if (idSolicitado !== idUsuarioSesion) {
    return res.status(403).json({
      mensaje: 'No tienes permiso para acceder a este recurso'
    });
  }

  const usuario = await db.query(
    'SELECT id, nombre, correo FROM usuarios WHERE id = ?',
    [idSolicitado]
  );

  res.json(usuario);
});
```

En la versión segura, el servidor compara el ID solicitado con el ID del usuario que inició sesión. Si no coinciden, se bloquea la solicitud con un error 403. Esto es importante porque los permisos nunca deben depender solo del frontend; siempre deben validarse en el backend.

2. A05:2025 — Injection

Código vulnerable

```
// Ejemplo vulnerable: consulta SQL construida con texto directo del usuario
app.post('/login', async (req, res) => {
  const { correo, contrasena } = req.body;

  const consulta = `
    SELECT * FROM usuarios
    WHERE correo = '${correo}'
    AND contrasena = '${contrasena}'
  `;

  const resultado = await db.query(consulta);

  if (resultado.length > 0) {
    res.json({ mensaje: 'Inicio de sesión correcto' });
  } else {
    res.status(401).json({ mensaje: 'Credenciales incorrectas' });
  }
});
```

El problema es que los datos ingresados por el usuario se pegan directamente dentro de la consulta SQL. Si alguien escribe texto malicioso en el campo de correo o contraseña, podría alterar la consulta original. Esto puede permitir accesos no autorizados o consulta de información sensible.

Código seguro

```
// Ejemplo seguro: consulta parametrizada
app.post('/login', async (req, res) => {
  const { correo, contraseña } = req.body;

  const resultado = await db.query(
    'SELECT * FROM usuarios WHERE correo = ? AND contraseña = ?',
    [correo, contraseña]
  );

  if (resultado.length > 0) {
    res.json({ mensaje: 'Inicio de sesión correcto' });
  } else {
    res.status(401).json({ mensaje: 'Credenciales incorrectas' });
  }
});
```

En la versión segura se usan consultas parametrizadas. Esto evita que los datos del usuario sean interpretados como parte del comando SQL. En lugar de construir la consulta manualmente, los valores se envían por separado y la base de datos los trata como datos, no como instrucciones.

3. A07:2025 — Authentication Failures

Código vulnerable

```
// Ejemplo vulnerable: contraseña guardada y comparada en texto plano
app.post('/login', async (req, res) => {
  const { correo, contraseña } = req.body;

  const usuario = await db.query(
    'SELECT * FROM usuarios WHERE correo = ?',
    [correo]
  );

  if (!usuario || usuario.contrasena === contraseña) {
    return res.json({
      mensaje: 'Inicio de sesión correcto'
    });
  }

  res.status(401).json({
    mensaje: 'Credenciales incorrectas'
  });
});
```

El problema es que la contraseña se compara en texto plano. Si la base de datos se filtra, las contraseñas quedarían expuestas directamente. Además, el código tiene una condición lógica peligrosa: permite iniciar sesión si no existe usuario o si la contraseña coincide, lo cual puede provocar comportamientos inseguros.

Código seguro

```
const bcrypt = require('bcrypt');

// Ejemplo seguro: contraseña protegida con hash
app.post('/login', async (req, res) => {
  const { correo, contrasena } = req.body;

  const usuario = await db.query(
    'SELECT * FROM usuarios WHERE correo = ?',
    [correo]
  );

  if (!usuario) {
    return res.status(401).json({
      mensaje: 'Credenciales incorrectas'
    });
  }

  const contrasenaValida = await bcrypt.compare(
    contrasena,
    usuario.contrasena_hash
  );

  if (!contrasenaValida) {
    return res.status(401).json({
      mensaje: 'Credenciales incorrectas'
    });
  }

  res.json({
    mensaje: 'Inicio de sesión correcto'
  });
});
```

En la versión segura, la contraseña no se guarda ni se compara en texto plano. Se usa un hash generado previamente con una librería como bcrypt. Cuando el usuario intenta iniciar sesión, se compara la contraseña ingresada contra el hash almacenado. Además, se evita revelar si el correo existe o no, usando siempre el mismo mensaje genérico de error.

Estos ejemplos muestran que muchas vulnerabilidades nacen de errores comunes durante el desarrollo: confiar demasiado en los datos del usuario, no validar permisos en el servidor o manejar mal las credenciales. Aplicar controles básicos como validación de autorización, consultas parametrizadas y hashing de contraseñas reduce de manera importante el riesgo de ataques reales.

Ranking personal de vulnerabilidades más críticas

Puesto	Vulnerabilidad	Justificación
1	A01:2025 — Broken Access Control	La considero la más crítica porque puede permitir que un usuario acceda a información o funciones que no le corresponden. Además, OWASP indica que mantiene el puesto número uno y que fue encontrada en el 100% de las aplicaciones probadas dentro de los datos considerados, lo que muestra que es un problema muy común.
2	A05:2025 — Injection	Es una de las más peligrosas porque una entrada mal validada puede terminar alterando consultas, comandos o procesos internos. OWASP explica que ocurre cuando datos no confiables llegan a un intérprete y este los ejecuta como instrucciones, lo que puede causar robo de información o control del sistema.
3	A03:2025 — Software Supply Chain Failures	La pongo en tercer lugar porque hoy muchas aplicaciones dependen de paquetes, librerías, APIs y servicios externos. Un solo componente comprometido puede afectar a muchos sistemas al mismo tiempo, como se vio en casos reales de cadena de suministro.
4	A07:2025 — Authentication Failures	Es crítica porque si falla la autenticación, el atacante puede entrar como si fuera un usuario legítimo. OWASP mantiene esta categoría en el puesto siete e incluye problemas como credenciales embebidas, autenticación incorrecta y fijación de sesión.
5	A02:2025 — Security Misconfiguration	La incluyo porque muchas veces no depende de código complejo, sino de errores básicos: paneles expuestos, permisos mal configurados, errores visibles o servicios innecesarios activos. Es peligrosa porque puede pasar desapercibida y abrir la puerta a ataques más graves.

Conclusión

Conocer el OWASP Top 10 es importante para cualquier desarrollador porque permite entender que la seguridad no es algo que se agrega hasta el final del proyecto, sino una parte que debe considerarse desde el diseño, la programación, las pruebas y el despliegue. Muchas vulnerabilidades no aparecen por ataques demasiado avanzados, sino por errores comunes como no validar permisos, usar configuraciones inseguras, guardar mal las contraseñas, confiar demasiado en librerías externas o no registrar eventos importantes del sistema.

El OWASP Top 10:2025 ayuda a identificar los riesgos más críticos en aplicaciones web actuales, como el control de acceso roto, las malas configuraciones, las fallas en la cadena de suministro de software, la inyección y los errores de autenticación. OWASP lo presenta como un documento de concientización estándar para desarrolladores y seguridad de aplicaciones web, además de confirmar que la versión vigente es la de 2025.

Como desarrollador, este conocimiento se puede aplicar revisando desde el inicio qué datos maneja la aplicación, quién debe tener acceso a cada función, cómo se protegen las credenciales, qué dependencias se instalan y cómo se registran los eventos importantes. También permite escribir código más seguro, usar consultas parametrizadas, proteger rutas del backend, validar entradas del usuario y evitar configuraciones por defecto.

En mi práctica profesional, aplicaría OWASP Top 10 como una guía de revisión antes de publicar cualquier sistema. No basta con que una aplicación funcione; también debe proteger la información de los usuarios, resistir errores comunes y facilitar la detección de incidentes. Al final, desarrollar software seguro no solo mejora la calidad técnica del proyecto, también demuestra responsabilidad profesional.

Referencias

OWASP Foundation. (2025). OWASP Top 10:2025. <https://owasp.org/Top10/2025/>

OWASP Foundation. (2025). OWASP Top Ten Web Application Security Risks.
<https://owasp.org/www-project-top-ten/>

OWASP Foundation. (2025). A01:2025 Broken Access Control.
https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/

OWASP Foundation. (2025). A02:2025 Security Misconfiguration.
https://owasp.org/Top10/2025/A02_2025-Security_Misconfiguration/

OWASP Foundation. (2025). A03:2025 Software Supply Chain Failures.
https://owasp.org/Top10/2025/A03_2025-Software_Supply_Chain_Failures/

OWASP Foundation. (2025). A04:2025 Cryptographic Failures.
https://owasp.org/Top10/2025/A04_2025-Cryptographic_Failures/

OWASP Foundation. (2025). A05:2025 Injection. https://owasp.org/Top10/2025/A05_2025-Injection/

OWASP Foundation. (2025). A06:2025 Insecure Design. https://owasp.org/Top10/2025/A06_2025-Insecure_Design/

OWASP Foundation. (2025). A07:2025 Authentication Failures.
https://owasp.org/Top10/2025/A07_2025-Authentication_Failures/

OWASP Foundation. (2025). A08:2025 Software or Data Integrity Failures.
https://owasp.org/Top10/2025/A08_2025-Software_or_Data_Integrity_Failures/

OWASP Foundation. (2025). A09:2025 Security Logging and Alerting Failures.
https://owasp.org/Top10/2025/A09_2025-Security_Logging_and_Alerting_Failures/

OWASP Foundation. (2025). A10:2025 Mishandling of Exceptional Conditions.
https://owasp.org/Top10/2025/A10_2025-Mishandling_of_Exceptional_Conditions/

Cybersecurity and Infrastructure Security Agency. (s. f.). Known Exploited Vulnerabilities Catalog.
<https://www.cisa.gov/known-exploited-vulnerabilities-catalog>

Cybersecurity and Infrastructure Security Agency. (2023). Threat Actors Exploit Atlassian Confluence CVE-2023-22515 for Initial Access to Networks. <https://www.cisa.gov/news-events/cybersecurity-advisories/aa23-289a>

Cybersecurity and Infrastructure Security Agency. (2023). Guidance Addressing Cisco IOS XE Web UI Vulnerabilities. <https://www.cisa.gov/guidance-addressing-cisco-ios-xe-web-ui-vulnerabilities>

National Institute of Standards and Technology. (s. f.). National Vulnerability Database.
<https://nvd.nist.gov/>

MITRE. (s. f.). Common Vulnerabilities and Exposures. <https://www.cve.org/>

MITRE. (s. f.). Common Weakness Enumeration. <https://cwe.mitre.org/>